When Data Quality Matters: Embedding-Informed Noise Injection and Cleaning for Vision Model Robustness

Alec Song, Bhavya Ranjan, Rushil Gupta, Saeed Arellano, Kushagra Kanaujia
The University of California at Santa Barbara / College of Engineering

March 17, 2026

1 Abstract

Large-scale vision models are typically trained on web-scale datasets whose labels contain pervasive noise, yet robustness research often models this noise as independent, uniformly random corruption that ignores dataset geometry and other quality issues [1, 2, 10, 11]. In this work, we build an end-to-end, data-centric pipeline that (i) uses Visual Layer's dataset-quality tooling to audit and clean public vision datasets, (ii) constructs embedding-aligned representations, and (iii) injects random, boundary-focused, and embedding-clustered label noise at controlled rates to study how the *topology* of corruption affects training and generalization in a ViT-based classifier. Our noise-injection framework operates directly in embedding space, concentrating mislabels either within local neighborhoods or near decision boundaries, and produces matched sets of clean and corrupted label tables that are consumed by a standardized COCO/ImageNet-style training pipeline. Across corruption levels of 5%, 15%, and 30%, we find that spatially coherent cluster noise is consistently less destructive than globally random noise, yielding faster convergence, lower training and validation loss, and smaller generalization gaps, while boundary-focused noise exhibits intermediate behavior that primarily degrades margin refinement rather than global representation quality [12, 14, 15]. These results demonstrate that corruption *structure* can matter as much as corruption rate, and they highlight that dataset-cleaning actions such as duplicate removal, leakage mitigation, and geometry-aware label audits can materially change both optimization efficiency and downstream robustness, providing a reproducible, dataset-engineering–oriented perspective on when data quality interventions are likely to pay off in modern vision pipelines [11, 14, 15].

2 Introduction

Modern computer vision systems are increasingly shaped by the scale and heterogeneity of their training corpora, yet the quality of supervision in those corpora remains difficult to quantify and easy to overlook. In practice, labels are produced via crowd work, heuristic pipelines, weak supervision, or web scraping, and therefore inevitably contain errors, ambiguities, and inconsistencies. A long line of work has shown that such label noise can degrade generalization, increase required sample complexity, and distort both training dynamics and evaluation conclusions [1, 2]. Recent data-centric analyses further argue that label issues are not a rare corner case but a pervasive property of widely used benchmarks, with demonstrable downstream effects on reported model quality [11, 10]. However, much of this work still models label noise as independent, uniformly random flips or simple class-conditional processes, even though real annotation errors are structured, clustered, and often entangled with other dataset-quality issues.

The literature commonly models label noise via explicit corruption processes. Standard settings include (i) symmetric noise, where labels are flipped uniformly at random; (ii) class-conditional (asymmetric) noise, where flips depend only on the true class; and (iii) more realistic instance-dependent noise, where corruption depends on the input (e.g., hard or ambiguous examples) [1, 3]. Under class-conditional assumptions, both theory and practice support approaches that correct the loss using an estimated noise transition matrix [3, 4]. In parallel, empirical work on deep networks demonstrated that over-parameterized models can fit random labels [5], motivating algorithmic defenses that attempt to prevent memorization of noisy samples. Prominent families include curriculum- and reweighting-based methods (e.g., MentorNet) [6], small-loss sample selection and co-training (e.g., Co-teaching) [7], loss-function design for robustness (e.g., generalized cross entropy) [8], and semi-supervised formulations that split data into presumed-clean vs. noisy subsets (e.g., DivideMix) [9]. Complementary dataset-centric methods aim to identify and prune likely label errors directly (e.g., Confident Learning and follow-on benchmark audits) [10, 11].

Despite the breadth of robust-training methods, two practical gaps remain for computer vision pretraining and dataset engineering. First, many benchmark studies and method comparisons still rely heavily on synthetic noise that is independent of the input (uniform flips) or depends only on class, even though real annotation mistakes are often structured (e.g., visually similar categories and clustered confusions), and may co-occur with other quality issues such as near-duplicates and train/validation leakage [11, 12]. Second, there is comparatively less controlled evidence that connects (a) measured, actionable dataset-quality interventions (e.g., duplicate removal, suspected mislabel pruning, outlier filtering) with (b) downstream model behavior across architectures and corruption regimes, using a reproducible pipeline that isolates data quality as the primary independent variable. Work in detection/segmentation has begun to explicitly study COCO-style annotation noise and its effects [12], but there is still no widely used, end-to-end experimental framework that links dataset auditing and cleaning to modern pretraining and evaluation under realistic noise.

This paper targets that gap by empirically quantifying how dataset-quality interventions and structured label corruption affect vision model performance. Our project is explicitly data-centric: we compare models trained on raw datasets, Visual Layer–cleaned datasets, and controlled variants with injected noise patterns designed to better approximate real-world errors (e.g., label flips among visually similar neighbors in embedding space and clustered label corruption) [2]. Concretely, we operationalize the intuition that many mislabels arise from near-class confusions (e.g., tiger vs. jaguar) by seeding noise using similarity structure in an embedding space rather than applying i.i.d. random flips, and we measure how performance degrades (and how much is recoverable) under increasing corruption. We treat common dataset-quality issues—such as near-duplicates, train–validation leakage, and misaligned preprocessing—as explicit experimental factors rather

than incidental cleanup, so that their impact on training can be measured alongside label-noise structure [11, 12]. We operationalize these ideas using Visual Layer's dataset-quality tooling to identify duplicates, leakage, and suspicious labels, producing both raw and cleaned versions of COCO that serve as bases for controlled noise injection. This end-to-end design supports our project goal of producing a reproducible research pipeline that (i) audits and curates public vision datasets, (ii) generates multiple noise-controlled dataset variants, (iii) trains modern architectures under matched hyperparameters, and (iv) reports statistically grounded comparisons across conditions.

In this work, we build an end-to-end, data-centric pipeline that (i) audits and cleans public vision datasets using Visual Layer and related tools, (ii) injects both random and embedding-informed structured label noise at controlled rates, and (iii) measures how these data-quality interventions and noise geometries affect the training and validation behavior of a modern ViT-based classifier on COCO. Our contributions are threefold: we introduce embedding-informed label-corruption schemes that concentrate noise among visually similar neighbors in feature space; we integrate these schemes into a reproducible dataset-engineering pipeline that also manipulates duplicates and leakage; and we empirically compare random versus structured noise across multiple corruption levels, highlighting when structured noise is less destructive for optimization and generalization [12, 14, 15]. By seeding noise using similarity relationships in embedding space, our framework explicitly treats the geometry of supervision as an experimental variable, enabling us to ask not only *how much* noise is present but *how it is organized* in feature space [14, 15].

In summary, the current landscape establishes that label noise is consequential and that many algorithmic mitigations can help under simplified corruption assumptions. However, the field still lacks systematic, reproducible evidence connecting realistic, structured noise and concrete dataset-cleaning actions to downstream performance for contemporary vision training pipelines. By combining data auditing/cleaning with embedding-informed noise injection and controlled training/evaluation, our work distinguishes itself from prior literature that primarily optimizes training objectives under idealized noise models, and instead provides actionable, dataset-engineering evidence about when and how data-quality interventions and the topology of label noise materially change model outcomes [11, 14, 15].

3 Methodology

3.1 Overall Experimental Design

Our methodology is organized as a data-centric, end-to-end pipeline that isolates dataset label quality as the primary independent variable. The pipeline consists of (1) constructing an embedding-aligned clean label table, (2) generating multiple noisy-label variants under controlled corruption schemes (random and embedding-structured), and (3) training and evaluating a fixed downstream vision model under matched hyperparameters while logging all runs for reproducibility. The codebase is split across two components: Alec's branch implements the noise-injection and dataset-variant generation utilities, while Bhavya's training pipeline consumes the resulting noisy label CSVs to train a vision classifier.

3.2 Data Representation and Alignment Assumptions

All noise generation methods operate on an *embedding-indexed* representation of the dataset. Specifically, we assume an embedding matrix

$$\mathbf{X} \in \mathbb{R}^{N \times D}$$

stored as a NumPy array (`embeddings.npy`), and a clean labels table stored as a CSV with N rows, where row i corresponds to embedding vector $\mathbf{X}_i$. The labels CSV includes (at minimum) an identifier column and a label column:

- `vector_id` (string identifier; configurable via `--id-column`)
- `label` (integer category id; configurable via `--label-column`)

For embedding-based corruption modes, the pipeline explicitly validates row alignment by asserting `X.shape[0] == len(labels_df)`. This alignment constraint is critical: all corruption choices are computed in embedding space and then applied by index to the label table.

3.3 Constructing Embeddings and Clean Label Tables

3.3.1 Exporting Embeddings to `.npy`

To enable fast nearest-neighbor queries, embeddings stored as per-sample `.pt` files are consolidated into a single matrix using `export_embeddings_npy.py`. The script reads a metadata CSV (default `metadata/embeddings.csv`) and infers relevant columns such as embedding path and label. For each row, it resolves the embedding file path robustly (handling relative paths and common repository layouts), loads the embedding tensor from `.pt` (supporting tensor or dictionary formats), flattens it to a 1D float vector, and stacks all vectors to form $\mathbf{X} \in \mathbb{R}^{N \times D}$. The output is:

- `embeddings.npy`: consolidated float32 embedding matrix.
- `labels_clean.csv`: an embedding-aligned label table containing `vector_id`, `label`, and `vector_path`; optional columns such as `image_id` and `object_id` are preserved if present in metadata.

3.3.2 COCO Single-Label Extraction for Image-Level Training

Because COCO is natively multi-object/multi-label at the image level, the training pipeline uses a deterministic single-label conversion based on the largest annotated bounding box. The utility `export_coco_single_label_csv.py` streams COCO samples and, for each image, selects the object with maximal area (largest $w \times h$) and records:

- `image_id`
- `object_id` (if available)
- `label` (COCO category id of the largest object)
- `file_name` (if available)

This provides an image-level label table compatible with single-label classification training.

3.4 Label Noise Injection

3.4.1 Noise Budget and Logging

All corruption methods operate under an *exact flip budget*. Given a dataset of size N and a desired noise level $\rho \in [0, 1]$, the target number of label changes is:

$$M = \lfloor \rho N \rfloor .$$

Each method produces:

- a noisy labels CSV identical in schema to the input labels CSV, with only the `label` column modified,
- a log CSV capturing each change with fields `index`, `original_label`, `new_label`, and a `reason` string describing the corruption mechanism (e.g., `random`, `border`, `border_fill_nn`, or `cluster_seed_<id>`).

Randomness is controlled via `--random-seed` to ensure reproducibility.

3.4.2 k-NN Infrastructure

Embedding-structured methods rely on k-nearest-neighbor search implemented with `sklearn.neighbors.NearestNe` Given $\mathbf{X}$, a distance metric (default cosine), and neighbor count k, the script computes neighbor indices and distances for each sample:

$$\mathcal{N}_k(i) = \mathrm{kNN}(\mathbf{X}_i).$$

A key helper operation is selecting the nearest neighbor with a *different* label:

$$j^\star = \arg \min_{j \in \mathcal{N}_k(i),\, y_j \neq y_i} d(\mathbf{X}_i, \mathbf{X}_j),$$

used to define label flips that induce a true change.

3.4.3 Random Noise (`mode=random`)

Random noise flips labels for M randomly chosen indices. For each selected index i, a new label is sampled uniformly from the set of observed labels excluding the current label:

$$\tilde{y}_i \sim \mathrm{Uniform}(\mathcal{Y} \setminus \{y_i\}).$$

This mode does not require embeddings, and serves as an i.i.d. corruption baseline.

3.4.4 Boundary-Focused Noise (`mode=border`)

Boundary noise prioritizes examples that appear near class boundaries in embedding space. For each sample i, the script computes a *margin hardness score* using neighbor distances:

$$\begin{aligned} d_{\mathrm{same}}(i) &= \min_{j \in \mathcal{N}_{k_b}(i),\, y_j = y_i} d(\mathbf{X}_i, \mathbf{X}_j), \\ d_{\mathrm{diff}}(i) &= \min_{j \in \mathcal{N}_{k_b}(i),\, y_j \neq y_i} d(\mathbf{X}_i, \mathbf{X}_j), \\ s(i) &= d_{\mathrm{diff}}(i) - d_{\mathrm{same}}(i). \end{aligned}$$

Smaller $s(i)$ indicates that the nearest different-class neighbor is comparably close to the nearest same-class neighbor, i.e., a boundary-like or ambiguous point. The algorithm sorts indices by increasing $s(i)$ and retains the top fraction α (`--boundary-top-frac`) as a boundary candidate pool. It then iterates through this pool in random order, flipping each selected index i to the label of its nearest different-label neighbor under a separate neighbor query size (`--nn-k`):

$$\tilde{y}_i \leftarrow y_{j^\star(i)}.$$

If boundary candidates are insufficient to reach the budget M, the method performs a *fill phase* that continues flipping non-changed indices from the full dataset using the same nearest-different-label rule (logged as `border_fill_nn`).

3.4.5 Cluster Noise (`mode=cluster`)

Cluster noise introduces structured, locally coherent corruption by flipping groups of nearby samples together. The method proceeds as follows:

1. Randomly permute indices and iterate over potential seed points.
2. For each seed s that has not been changed, select a *single* target label based on the seed's nearest different-label neighbor:
$$\ell_s \leftarrow y_{j^\star(s)}.$$
3. Define a local cluster as the seed's nearest neighbors (including itself), truncated to size C (`--cluster-size`):
$$\mathcal{C}(s) = \{\text{ first } C \text{ indices in } \mathcal{N}_k(s)\}.$$
4. Flip as many non-changed members of $\mathcal{C}(s)$ as needed until the global budget M is reached:
$$\tilde{y}_i \leftarrow \ell_s \quad \forall i \in \mathcal{C}(s) \text{ (until budget met)}.$$

This produces contiguous regions in embedding space where multiple neighboring samples share the same incorrect label, approximating real-world label noise that arises from systematic confusions rather than independent flips. Each flip is logged with a seed-specific reason string (e.g., `cluster_seed_1234`) to enable post-hoc diagnostics.

3.4.6 Batch Generation of Noise Variants

To standardize dataset variant creation, `run_noise_variants.sh` demonstrates generating multiple corruption modes with fixed hyperparameters (cosine distance, fixed seed). This produces matched sets of random, cluster, and boundary label files that can be consumed directly by training scripts.

3.5 Downstream Training and Evaluation Pipeline

3.5.1 Dataset and Sampling

Bhavya's training pipeline (`train_coco_noise.py`) trains a single-label image classifier using the Hugging Face COCO dataset (`detection-datasets/coco`) in streaming mode. The train stream is shuffled with a finite buffer (`buffer_size=10,000`) and fixed seed for determinism. Each streamed sample provides an image and per-object annotations. The pipeline converts COCO to single-label classification by selecting the largest object in each image and cropping the image to that bounding box before applying transforms.

3.5.2 Noisy Label Injection at Train Time

Training supports noisy labels by loading a precomputed noisy CSV (e.g., `alec/labels/labels_border_10.csv`) and constructing a lookup map `image_id` $\rightarrow$ `label`. For each training sample with image id u:

$$y(u) = \begin{cases} \tilde{y}(u) & \text{if } u \in \text{noisy lookup}, \\ y_{\text{clean}}(u) & \text{otherwise.} \end{cases}$$

The script logs the number of noisy vs. clean labels used per epoch, enabling verification that the intended corruption rate is being applied (subject to lookup coverage and dataset filtering).

3.5.3 Model Architecture

The model is instantiated with `timm.create_model` using a Vision Transformer backbone (`vit_base_patch32_224`) configured for 80 COCO categories (`num_classes=80`). Regularization includes nonzero dropout (`drop_rate=0.2`) and stochastic depth (`drop_path_rate=0.1`). The model is trained from scratch (`pretrained=False`).

3.5.4 Optimization and Training Procedure

Training uses cross-entropy loss with label smoothing (`label_smoothing=0.1`), AdamW optimization, cosine annealing learning rate schedule, and mixed precision training:

- Image resolution: 224×224.
- Batch size: 64 (manual batch accumulation from the stream).
- Epochs: 30.
- Optimizer: AdamW with learning rate 5×10^{-5} and weight decay 10^{-2}.
- Scheduler: CosineAnnealingLR with $T_{\max} = 30$.
- Mixed precision: `torch.amp.autocast` with `GradScaler`.

The training loop processes streamed samples until a fixed cap per epoch (`total >= 118,000`) to approximate a consistent epoch size, and validation similarly caps evaluation to a fixed number of examples (`val_total >= 5,000`).

3.5.5 Data Augmentation and Normalization

Training and validation apply standard ImageNet-style normalization. Training additionally applies stochastic augmentations:

- Training transforms: random resized crop, horizontal flip, color jitter, normalization.
- Validation transforms: resize to 224×224, normalization.

All samples are converted to RGB after cropping.

3.5.6 Experiment Tracking and Checkpointing

The pipeline uses Weights & Biases (W&B) for experiment tracking (`wandb.init`) and logs per-epoch metrics including train loss, validation loss, learning rate, and counts of noisy vs. clean labels used. Model weights are saved each epoch to a checkpoint file and uploaded to W&B for run-level artifact tracking.

3.6 Reproducibility Considerations

Reproducibility is supported through: (1) deterministic corruption via explicit random seeds in noise generation, (2) explicit row-alignment checks between embeddings and label tables, (3) logged per-change corruption records, and (4) centralized run logging of all training hyperparameters and noisy label sources (CSV paths) via W&B configuration. Together, these design choices allow exact regeneration of dataset variants and matching of downstream runs to their corresponding data conditions.

4 Results

We evaluate how label-noise *structure* (random vs. embedding-clustered) affects optimization and generalization when training a ViT-B/32 model on COCO. Unless otherwise noted, all runs are trained for approximately 50 epochs and compared against a clean COCO baseline (Raw). Noise is injected at controlled rates of 5%, 15%, and 30% under three corruption regimes:

- **Random noise:** uniformly random label flips.
- **Cluster noise:** spatially coherent label flips within embedding-defined local neighborhoods.
- **Border noise:** label flips concentrated near embedding decision boundaries, targeting ambiguous or low-margin samples.

4.1 Training Loss Dynamics

Figure 1 summarizes training loss trajectories across random noise levels. Figure 3 summarizes training loss trajectories across cluster noise levels.

4.1.1 Clean Baseline

The clean dataset exhibits the fastest and deepest convergence, with rapid early loss descent, a smooth monotonic decline, and a final training loss of approximately 2.59. This serves as an upper bound on optimization efficiency under correct supervision.

4.1.2 Random Noise

Random noise yields a consistent degradation pattern characterized by higher initial loss, slower convergence, and a higher asymptotic plateau. The 5%, 15%, and 30% curves are strongly separated throughout training. At 30% random noise, training loss remains elevated (approximately 3.77) and convergence is substantially slowed, suggesting widespread gradient corruption and underfitting relative to lower noise levels. Overall, random noise exhibits near-linear degradation with respect to noise magnitude.

4.1.3 Cluster Noise

In contrast, cluster noise produces tightly grouped training-loss curves across 5%, 15%, and 30% corruption. Degradation scales smoothly and sub-linearly: even at 30% cluster noise, training loss (approximately 3.38) remains substantially below the 30% random-noise condition. The training dynamics under cluster noise resemble scaled variants of a common trajectory rather than qualitatively different optimization regimes, suggesting that structured corruption perturbs local class neighborhoods while preserving global feature organization.

4.1.4 Border Noise

Border noise produces training dynamics intermediate between cluster and random corruption. Because corruption is concentrated near embedding decision boundaries, early optimization remains relatively stable compared to random noise, but asymptotic loss exceeds that of cluster noise at comparable corruption levels. At 30% border noise, training loss remains below the corresponding random-noise condition but above cluster noise. This suggests that corruption targeted at ambiguous boundary samples perturbs margin refinement rather than globally fragmenting the feature

space. Optimization remains coherent in early epochs but experiences reduced late-stage boundary sharpening.

4.2 Validation Loss Dynamics

Figure 2 summarizes validation loss trajectories across random noise levels. Figure 4 summarizes validation loss trajectories across cluster noise levels.

4.2.1 Clean Dataset

Clean training yields the lowest validation loss (approximately 2.65) and the largest margin relative to noisy conditions. The generalization gap is stable and minimal.

4.2.2 Random Noise

Random noise significantly harms validation performance, with clear monotonic degradation as noise increases. At 30% random noise, validation loss converges near approximately 3.25, and the separation between 5%, 15%, and 30% remains pronounced. The widening gap relative to the clean baseline is consistent with impaired generalization due to globally inconsistent supervision.

4.2.3 Cluster Noise

Cluster noise again exhibits substantially tighter grouping across corruption levels and lower degradation than random noise. Even at 30% cluster noise, validation loss (approximately 3.16–3.18) remains noticeably better than the corresponding random-noise condition. That the validation trajectories track training behavior closely suggests representation learning is less disrupted by spatially coherent mislabeling than by uniformly random label flips.

4.2.4 Border Noise

Validation behavior under border noise mirrors training trends. Final validation loss remains consistently lower than random noise at equivalent corruption rates but does not achieve the stability of cluster noise. This indicates that structured corruption near class boundaries harms generalization primarily through margin distortion rather than widespread representational disruption. Compared to random noise, border noise preserves more coherent global structure, but because it targets difficult examples, it reduces the model's ability to confidently separate adjacent classes.

4.3 Comparative Analysis

4.3.1 Convergence Speed

Clear separation emerges in early-epoch optimization dynamics. Clean training converges fastest, followed by cluster noise, border noise, and finally random noise. Over the first 5–10 epochs, random noise exhibits the slowest loss descent, indicating disrupted gradient alignment.

Cluster noise preserves convergence shape most closely to clean training. Border noise remains closer to cluster than random in early descent but diverges modestly in later epochs as boundary refinement becomes impaired. These results indicate that corruption structure directly affects optimization efficiency.

4.3.2 Asymptotic Performance

Across all regimes, the ordering of final performance is consistent:

$$\text{Clean} < 5\%\ \text{Cluster} \approx 5\%\ \text{Random} < 15\%\ \text{Cluster} < 15\%\ \text{Random} < 30\%\ \text{Cluster} \ll 30\%\ \text{Random}.$$

The separation between 30% cluster and 30% random noise is particularly large, indicating that corruption *structure* can dominate corruption *rate* at high noise levels.

4.3.3 Generalization Gap

We quantify the generalization gap as:

$$\text{Gap} = \text{Validation Loss} - \text{Training Loss}.$$

Across all corruption levels, random noise produces the largest gap, particularly at 30% corruption. Cluster noise maintains the smallest gap among noisy conditions, while border noise remains intermediate.

Importantly, gap widening under random noise accelerates in later epochs, suggesting increasing memorization pressure under globally inconsistent supervision. In contrast, cluster noise exhibits a more stable train–validation divergence, consistent with locally coherent (though shifted) decision regions. Border noise increases gap primarily through degraded boundary calibration rather than global instability.

4.4 Interpretation in the Capstone Context

These results support the core hypothesis that not all label noise is equally destructive and that structured corruption interacts differently with modern representation learning.

1. **Structured noise is more optimization-friendly than random noise.** Cluster noise preserves convergence shape and yields sub-linear degradation with respect to the corruption rate, whereas random noise disrupts optimization globally.

2. **Geometry matters.** Because cluster noise respects embedding proximity, corruption is spatially localized: global class manifolds remain more intact, and decision boundaries shift locally rather than fragmenting across the entire feature space.

3. **Noise magnitude alone is insufficient.** Two datasets with identical corruption rates (e.g., 30%) can produce dramatically different outcomes depending on corruption structure. This has practical implications for dataset auditing and cleaning, robust training strategy selection, and the design of synthetic noise models used in robustness research.

a

5 Discussion

5.1 Significance of the Results

This study investigated whether the *structure* of label noise—not merely its magnitude—influences optimization dynamics and generalization behavior in transformer-based vision models. The results provide clear empirical evidence that it does. Across all corruption levels (5%, 15%, 30%), spatially

structured cluster noise consistently produced lower training and validation loss than random noise at equivalent corruption rates, while border noise exhibited intermediate behavior.

Moreover, degradation under cluster noise scaled more smoothly and sub-linearly, whereas random noise exhibited sharper and more linear performance decline. Border noise revealed a distinct third regime: corruption concentrated near embedding decision boundaries preserved much of early-stage optimization efficiency but impaired late-stage margin refinement. These findings directly support the central research question: label noise is not uniformly destructive; its geometric structure materially affects learning dynamics.

5.2 Summary of Key Findings

Three major findings emerge from the results:

1. **Structured noise is less disruptive than random noise.** At 30% corruption, random noise led to substantially higher training and validation loss than both cluster and border noise, indicating that uniformly distributed label flips inject globally inconsistent gradient signals and impair optimization across the feature space. In contrast, cluster noise (defined within embedding-local neighborhoods) preserves much of the global structure of the data manifold; even when labels are incorrect, they remain spatially coherent, and the model appears better able to adapt to localized corruption. Border noise, which targets low-margin or ambiguous samples, disrupts decision-boundary calibration rather than fragmenting the entire representation space.

2. **Degradation depends on corruption topology.** Performance under random noise degrades roughly proportionally to noise magnitude. Under cluster noise, increasing corruption from 15% to 30% produces comparatively modest additional harm, suggesting diminishing marginal disruption once a local region of the feature space is corrupted. Border noise exhibits intermediate scaling: because corruption is concentrated near class boundaries, increasing corruption primarily affects margin sharpness rather than global representation learning. This indicates that corruption location within the manifold is as important as corruption rate.

3. **Random noise amplifies generalization harm.** Validation loss under random noise diverges more strongly from the clean baseline than under cluster or border noise. The larger generalization gap suggests that random noise increases gradient variance and introduces contradictory supervisory signals across the representation space. Cluster noise produces more stable train–validation behavior, consistent with locally coherent (though shifted) decision regions. Border noise increases generalization error primarily through degraded boundary calibration, indicating a different failure mode focused on margin distortion rather than global instability.

5.3 Relation to Prior Literature

Prior work in robust learning has often treated label noise as a scalar quantity—defined by a corruption percentage—and modeled it as uniformly random (Patrini et al., 2017; Zhang et al., 2017). However, real-world dataset audits, including re-evaluations of ImageNet (Recht et al., 2019), demonstrate that mislabeling is often systematic rather than uniform. Errors frequently cluster around visually similar or semantically adjacent categories rather than occurring as independent random flips.

The present findings extend this perspective by distinguishing between *manifold-internal* corruption (cluster noise) and *boundary-targeted* corruption (border noise). Under manifold-based

interpretations of deep learning (Bengio et al., 2013; Belkin et al., 2019), data lie on structured low-dimensional subspaces embedded in high-dimensional feature space. Random noise disrupts manifold alignment globally by injecting inconsistent supervisory signals across the representation space. Cluster noise perturbs local regions while preserving broader geometric organization. Border noise, by contrast, primarily distorts decision margins, altering class separation without fully fragmenting the underlying manifold.

Together, these results reinforce emerging perspectives that data geometry and noise topology are critical to understanding robustness. Not all structured noise behaves identically: corruption that respects embedding neighborhoods differs meaningfully from corruption concentrated near class boundaries, and both differ fundamentally from globally random label flips.

5.4 Implications

5.4.1 Noise Magnitude Is an Incomplete Descriptor

Two datasets with identical corruption rates (e.g., 30%) can produce substantially different optimization outcomes depending on how that corruption is distributed. This challenges common benchmarking practices that rely solely on a noise percentage.

5.4.2 Dataset Auditing Should Consider Spatial Structure

Dataset cleaning pipelines may benefit from detecting and correcting cluster-based label inconsistencies rather than focusing exclusively on random noise detection. Embedding-based clustering methods could serve as diagnostic tools for identifying systematic annotation errors.

5.4.3 Robust Training Strategies Should Be Structure-Aware

If structured noise is less harmful than random noise, training algorithms designed under uniform noise assumptions may mischaracterize real-world robustness challenges. Future robustness research should incorporate geometry-aware noise models.

5.4.4 Optimization Efficiency and Energy Implications

Because random noise slows early descent and elevates asymptotic loss, it effectively increases the epochs required to reach a fixed loss threshold. This implies greater computational and energy cost for equivalent performance. Cluster noise, by preserving optimization efficiency, reduces effective compute burden relative to random corruption. Border noise occupies an intermediate position: early efficiency is preserved, but later-stage refinement requires additional optimization effort.

Thus, corruption topology influences not only accuracy but also training efficiency and energy expenditure.

5.4.5 Dataset Geometry vs Model Capacity

The results suggest that model capacity interacts strongly with corruption topology. High-capacity transformer architectures are capable of partially memorizing random noise, but this memorization manifests as larger generalization gaps and unstable convergence.

Under cluster noise, capacity appears to adjust decision boundaries locally without globally fragmenting the representation space. Border noise highlights a third regime: capacity is sufficient to fit ambiguous samples but struggles to maintain margin sharpness.

These findings suggest that dataset structure may be as important as dataset cleanliness. Increasing model capacity cannot fully compensate for globally inconsistent supervision, but it can partially absorb structured corruption.

5.5 Limitations

Several limitations constrain the scope of this study:

- **Loss-based evaluation only.** The analysis relies on training and validation loss curves. Accuracy, mAP, and per-class performance were not evaluated, limiting interpretability of downstream task impact.
- **Single architecture.** Experiments were conducted using ViT-B/32. Different model families (e.g., CNNs) may respond differently to structured corruption.
- **Synthetic noise model.** Although cluster noise introduces structure, it remains synthetic; real-world annotation errors may exhibit more complex patterns.
- **No direct representation analysis.** The study infers geometric preservation indirectly through optimization behavior rather than directly measuring embedding drift or feature similarity.

5.6 Future Directions

Several promising avenues follow from these results:

- **Representation geometry analysis:** measure inter-class distances and embedding centroid drift to directly quantify geometric preservation.
- **Gradient variance measurement:** test whether random noise produces higher gradient variance than cluster noise, providing mechanistic confirmation.
- **Scaling across architectures:** compare CNNs and larger transformers to assess whether model capacity mediates sensitivity to structured corruption.
- **Real-world noise modeling:** simulate empirically observed dataset auditing patterns rather than uniform corruption rates.
- **Compute efficiency analysis:** quantify epochs-to-threshold loss under different noise regimes to connect corruption structure to training efficiency.

5.7 Broader Takeaway

The central insight of this study is that label noise topology reshapes the optimization landscape. Random corruption disrupts gradients globally, cluster corruption perturbs learning locally while preserving manifold geometry, and border corruption primarily distorts decision margins.

Robustness, therefore, is not determined solely by how much noise exists, but by where that noise lies relative to the data manifold and decision boundaries. Future robustness benchmarks and dataset auditing frameworks should explicitly account for corruption geometry rather than relying on scalar noise percentages alone.

References

[1] B. Frénay and M. Verleysen. Classification in the presence of label noise: a survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5):845–869, 2014.

[2] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee. Learning from noisy labels with deep neural networks: a survey. *IEEE Transactions on Neural Networks and Learning Systems* (survey preprint: arXiv:2007.08199), 2020.

[3] N. Natarajan, I. S. Dhillon, P. K. Ravikumar, and A. Tewari. Learning with noisy labels. In *NeurIPS*, 2013.

[4] G. Patrini, A. Rozza, A. K. Menon, R. Nock, and L. Qu. Making deep neural networks robust to label noise: a loss correction approach. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[5] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.

[6] L. Jiang, Z. Zhou, T. Leung, L.-J. Li, and L. Fei-Fei. MentorNet: Learning data-driven curriculum for very deep neural networks on corrupted labels. In *ICML*, 2018.

[7] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. Tsang, and M. Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *NeurIPS*, 2018.

[8] Z. Zhang and M. R. Sabuncu. Generalized cross entropy loss for training deep neural networks with noisy labels. In *NeurIPS*, 2018.

[9] J. Li, R. Socher, and S. C. H. Hoi. DivideMix: Learning with noisy labels as semi-supervised learning. In *ICLR*, 2020.

[10] C. G. Northcutt, L. Jiang, and I. L. Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research* (preprint: arXiv:1911.00068), 2019–2021.

[11] C. G. Northcutt, A. Athol, and I. L. Chuang. Pervasive label errors in test sets destabilize machine learning benchmarks. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[12] K. Kuba et al. Combating noisy labels in object detection datasets. *arXiv preprint* arXiv:2211.13993 (v3: 2023), 2023.

[13] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar. Do ImageNet classifiers generalize to ImageNet? In *International Conference on Machine Learning (ICML)*, 2019.

[14] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 35(8):1798–1828, 2013.

[15] M. Belkin, D. Hsu, S. Ma, and S. Mandal. Reconciling modern machine learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences (PNAS)*, 116(32):15849–15854, 2019.

Figures

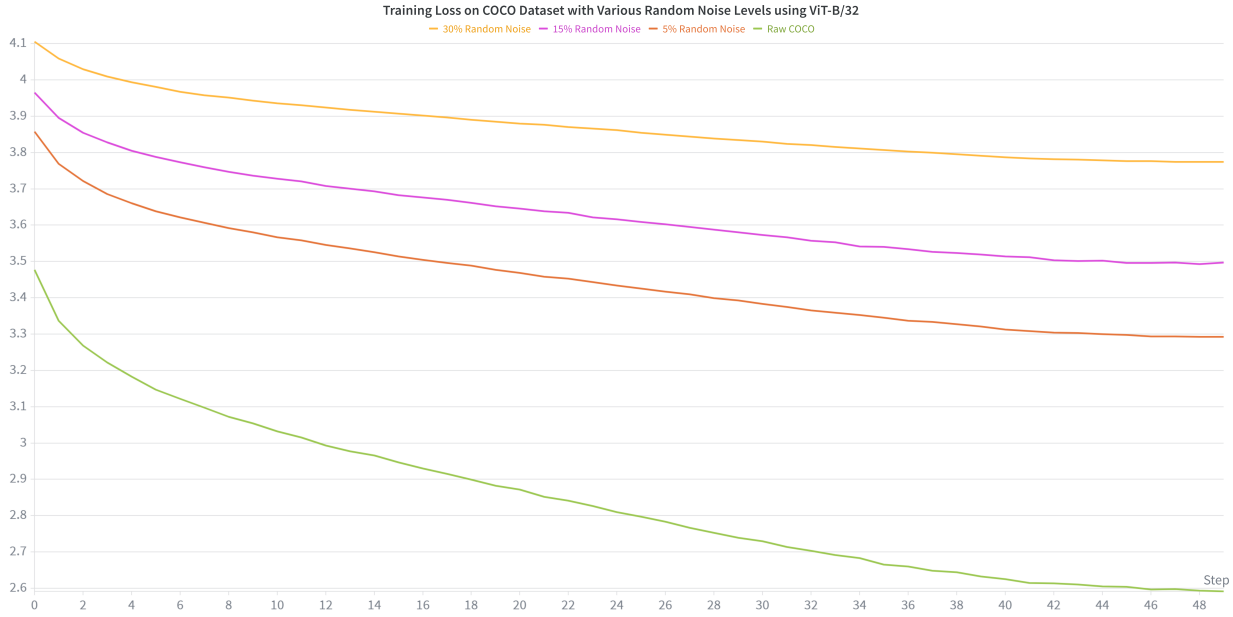

Figure 1: Training loss on COCO under **random** label noise (ViT-B/32).

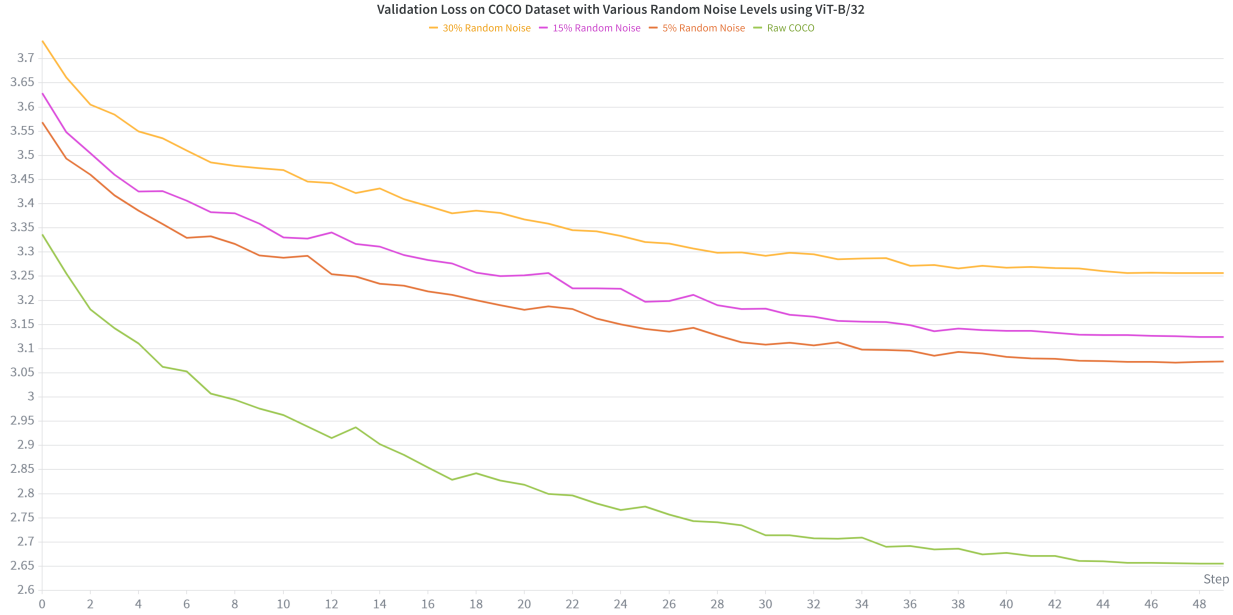

Figure 2: Validation loss on COCO under **random** label noise (ViT-B/32).

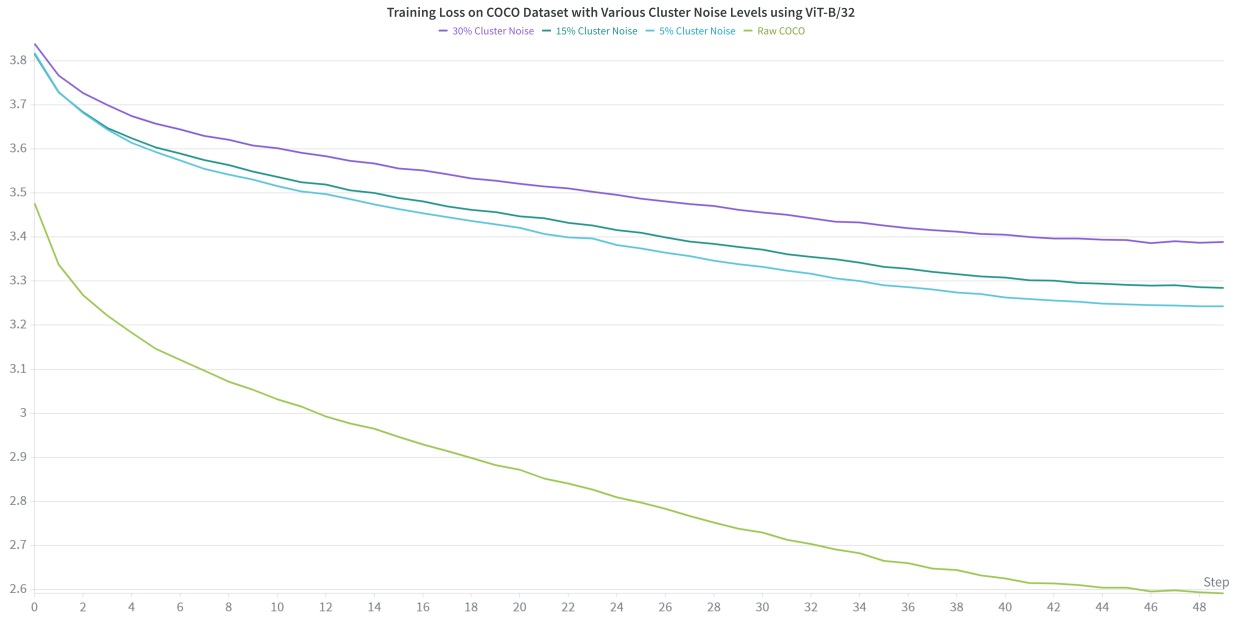

Figure 3: Training loss on COCO under **cluster** (embedding-structured) label noise (ViT-B/32).

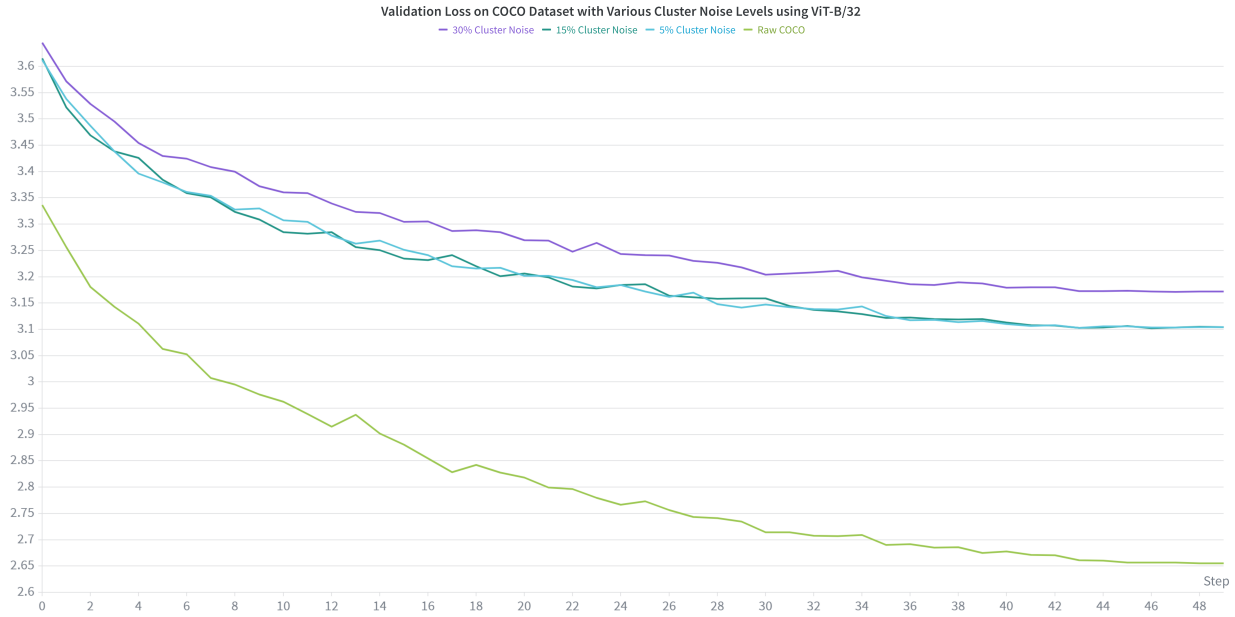

Figure 4: Validation loss on COCO under **cluster** (embedding-structured) label noise (ViT-B/32).

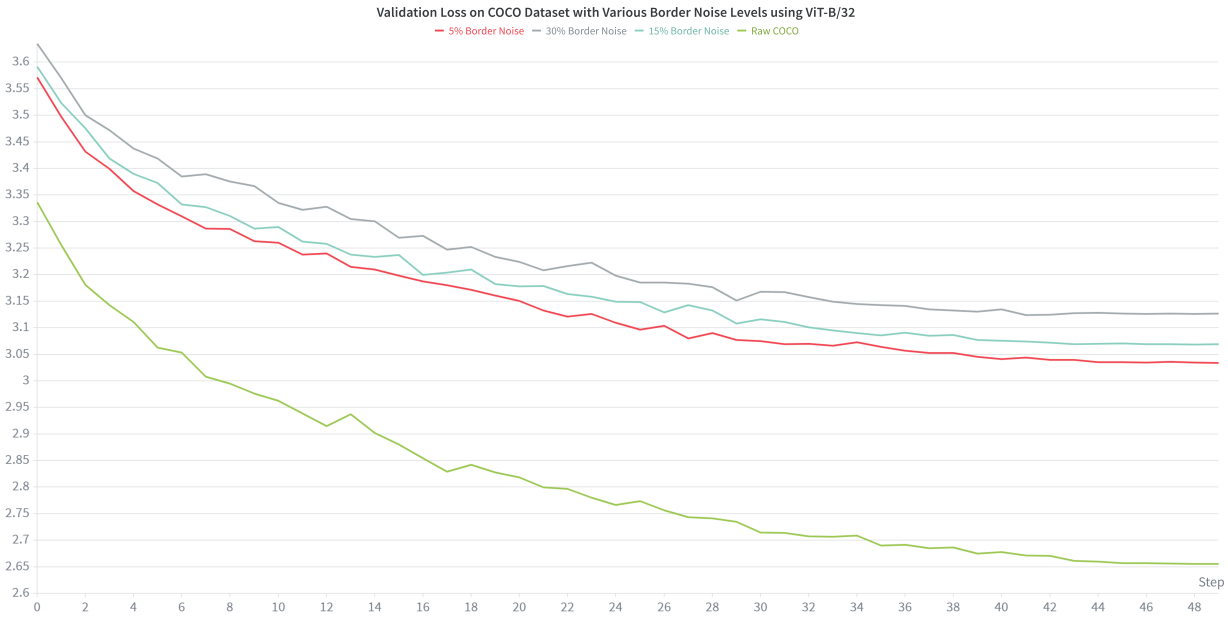

Figure 5: Validation loss on COCO under **border** (embedding-structured) label noise (ViT-B/32).

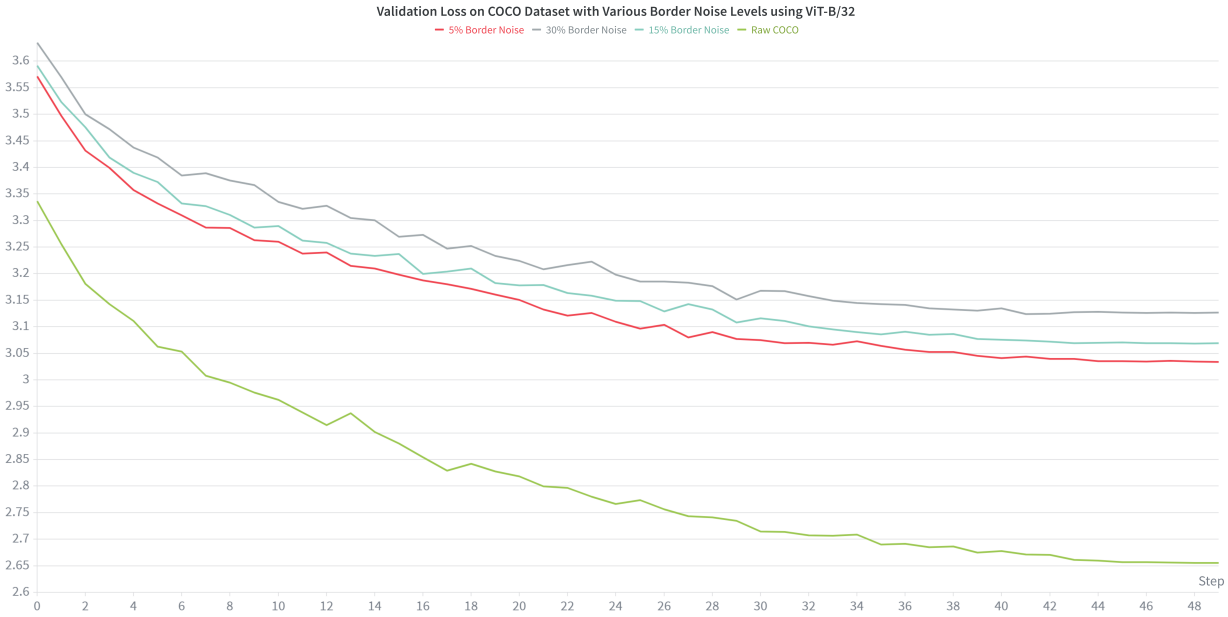

Figure 6: Validation loss on COCO under **border** (embedding-structured) label noise (ViT-B/32).